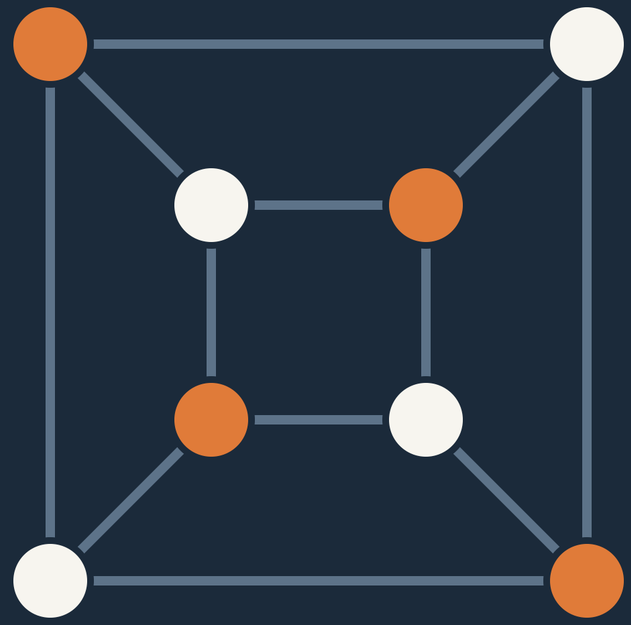
A GUIDED TOUR OF TWO PAPERS

Graph Isomorphism from First Principles

Symmetry, Algebra, and Algorithms

From colour refinement to an explicit quasi-polynomial bound, with examples you can run yourself



Christos I. Karatzas · Independent researcher, Greece

ROADMAP

Seven steps, from dots to the bound

1

Fundamentals

Graphs, symmetry, groups, orbits

2

Refinement

The standard tool, and where it fails

3

Refinement-derived algebra

Candidates, then linear equations

4

Practical solver

Non-abelian symmetry, cuts, Luks

5

Explicit bound

Giants, bits, conjecture G2

6

Evidence

Against Traces, hardest groups

7

Hands-on

Build, generate, run, test isomorphism; glossary of every term

THE CONTRIBUTION IN ONE SENTENCE

We use invariant refinement to expose algebraic structure, solve the resulting symmetry constraints, and combine that practical solver with a fully specified quasi-polynomial fallback and explicit branching-bit accounting.

The executable

Colouring, series solver, stabiliser chain, cut and star routes with Luks. Exact on every instance it answers; public code, graphs and logs.

The theoretical hybrid

The same stages plus Babai's giant case as last fallback. The bound counts all of them. The giant case is **specified, not implemented**: no instance reached it.

Plus two open conjectures, stated precisely: **Conjecture 11.1** (candidate sets are groups) and **G2** (the giant case is never needed).

FUNDAMENTALS

Graphs and isomorphism

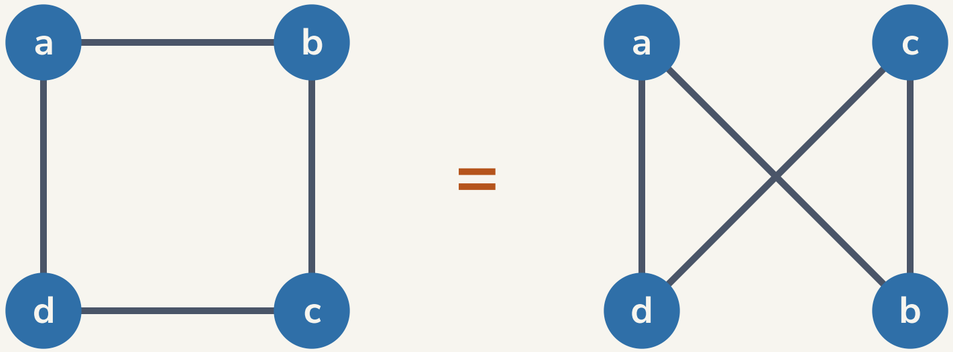
DEFINITION

A **graph** G = (V, E): a set V of **vertices** and a set E of **edges**, each edge a pair {u, v} of vertices. n = |V|.

DEFINITION

An **isomorphism** G → H is a renaming φ of the vertices (a bijection) such that {u, v} is an edge of G exactly when {φ(u), φ(v)} is an edge of H.

Graph isomorphism (GI): given G and H, decide whether an isomorphism G → H exists.



FUNDAMENTALS

Why it is hard: too many renamings

4 vertices

24

renamings to try

10 vertices

3,628,800

renamings to try

50 vertices

3×10^{64}

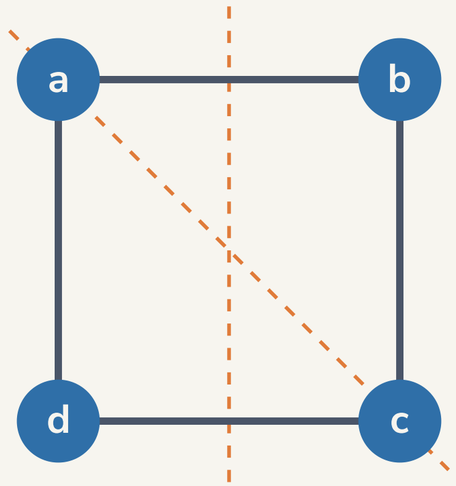
more than atoms in the Earth

With n vertices there are n! = 1 × 2 × … × n renamings. Trying them all is hopeless, so every good algorithm uses the **structure** of the graph instead.

Status: no polynomial-time algorithm is known. GI is not known to be NP-complete, and such a result would have major complexity-theoretic consequences (the polynomial hierarchy would collapse). Babai (2015) gave a quasi-polynomial algorithm; more on that in part 5.

FUNDAMENTALS

A symmetry is an automorphism



DEFINITION

An **automorphism** of G is an isomorphism G → G: a renaming of the vertices that maps every edge to an edge. Aut(G) is the set of all of them.

NOTATION

A renaming is a **permutation**, written by its images: a → b, b → c, c → d, d → a.

Check: that quarter turn sends ab → bc, bc → cd, cd → da, da → ab. All edges, so it is an automorphism. The square has 8: four turns (one is “do nothing”) and four mirrors.

FUNDAMENTALS

Symmetries form a group

DEFINITION

A **permutation group** is a set of permutations that contains “do nothing” (the **identity**), and with any two members their composition and inverses. Its size is its **order**.

DEFINITION

A set S **generates** a group if every member is a composition of members of S. S is a list of **generators**.

Aut(G) is a group: compose two symmetries and you get a symmetry. Algorithms store **generators**, never the whole list.

Why generators

7.0×10^{70}

automorphisms of the benchmark graph CHH(7-7)-2. No computer can list them; a short generator list describes all of them.

FUNDAMENTALS · EXAMPLE

Two generators give all 8 symmetries

r = quarter turn: a → b, b → c, c → d, d → a

s = mirror on the diagonal ac: b ↔ d, a and c stay

Write **rs** for “first s, then r”, and r^2 for r twice.

Every symmetry is r^i or $r^i s$ with i = 0, 1, 2, 3: 4 × 2 = 8. Two generators describe the whole group.

word	a →	b →	c →	d →	what it is
1	a	b	c	d	identity
r	b	c	d	a	turn 90°
r^2	c	d	a	b	turn 180°
r^3	d	a	b	c	turn 270°
s	a	d	c	b	mirror on ac
rs	b	a	d	c	mirror, swaps a ↔ b
r^2s	c	b	a	d	mirror on bd
r^3s	d	c	b	a	mirror, swaps a ↔ d

FUNDAMENTALS

Orbits and stabilisers

DEFINITION

The **orbit** of a vertex v: all vertices that some symmetry sends v to.

DEFINITION

The **stabiliser** of v: the symmetries that keep v in place. It is a group too.

ORBIT-STABILISER THEOREM

$$|\mathrm{Aut}(G)| = |\text{orbit of } v| \times |\text{stabiliser of } v|$$

On the square, v = a

Orbit {a, b, c, d}, reached by 1, r, r^2, r^3. Stabiliser {1, s}: from the table, only those keep a. So $4 \times 2 = 8$. ✓

Repeat: a stabiliser chain

Fix a, then b: inside {1, s} the orbit of b is {b, d}; fixing b leaves only 1. $|\mathrm{Aut}| = 4 \times 2 \times 1$.

FUNDAMENTALS

One computation answers both questions

DEFINITION

The **disjoint union** G + H: both graphs side by side as one graph, no edges between them. A graph is **connected** if it is in one piece.

Fact. For connected G and H: G and H are isomorphic exactly when some automorphism of G + H maps a vertex of G into H. Restricted to G, it **is** an isomorphism.

Counting is enough: |Aut(G + H)| = 2·|Aut G|·|Aut H| if isomorphic, |Aut G|·|Aut H| if not.

Example: an S_3-CFI graph G with |Aut G| = 1,296

pair	|Aut(G + H)|	answer
G and a shuffled copy	3,359,232	isomorphic
G and its twisted version	559,872	not isomorphic

3,359,232 = 2 × 1,296²; 559,872 = 1,296 × 432 (the twist has 432). You run this test at the end.

COLOUR REFINEMENT

Colour refinement: let the neighbours vote

DEFINITION

A **colouring** gives each vertex a colour. A **class** is the set of vertices of one colour.

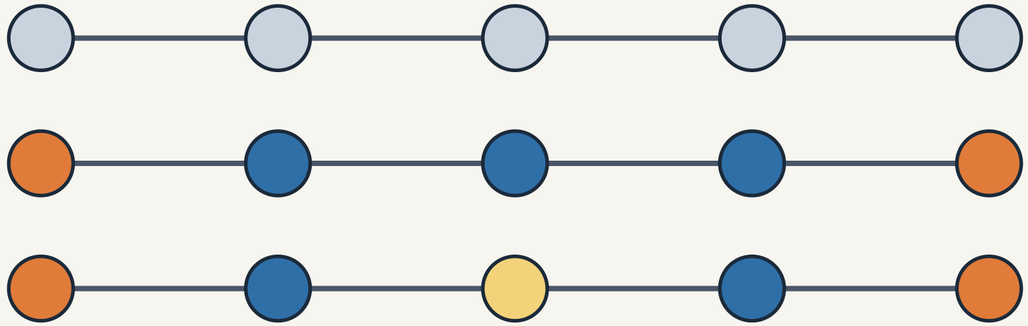
DEFINITION

Refinement round: new colour = old colour + the multiset of neighbours' colours. **Stable**: a round changes nothing.

Start: one colour

Round 1: 1 vs 2 neighbours

Round 2: stable, 3 classes



Classes at the end: **ends**, **next to an end**, **middle**. Also called 1-WL (Weisfeiler-Leman); it runs in O((n + m) log n) time, m = number of edges.

COLOUR REFINEMENT

Refinement never lies, but may not tell all

DEFINITION

A colouring is **invariant** if it is computed from the edges alone, never from vertex names. Then every automorphism φ keeps colours: colour(φ(v)) = colour(v).

What it guarantees

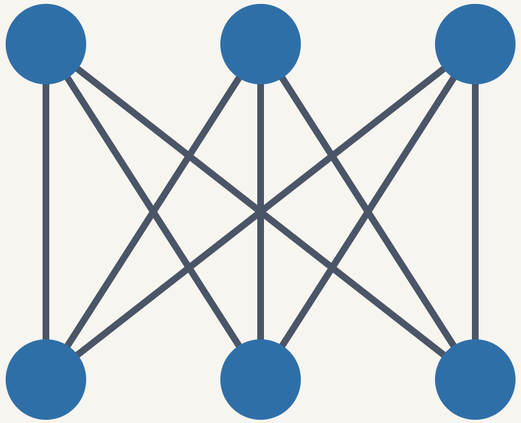
Each class is a union of orbits: a symmetry moves a vertex only within its class. If every class is one vertex, Aut(G) is just the identity.

What it does not

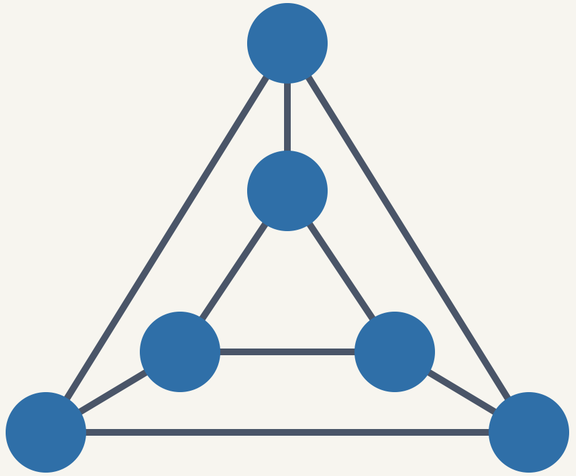
Two vertices of one class need not share an orbit. A class can look far more symmetric than the graph is. All the difficulty lives in this gap.

COLOUR REFINEMENT

Where refinement is blind



$K_{3,3}$: no triangles

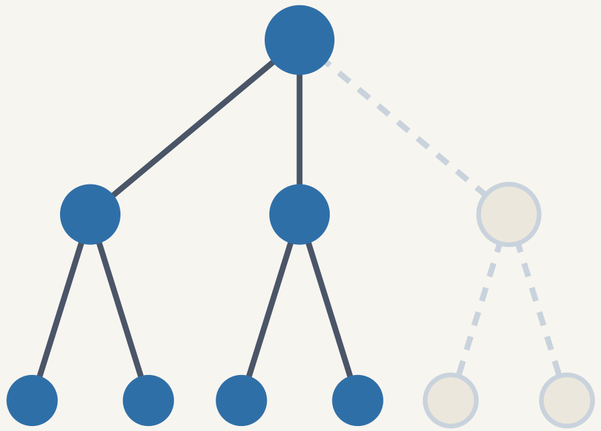


Prism: two triangles

Both have 6 vertices, each with 3 neighbours. Refinement gives every vertex the same colour in both graphs, yet they are **not** isomorphic.

COLOUR REFINEMENT

Individualise and refine



DEFINITIONS

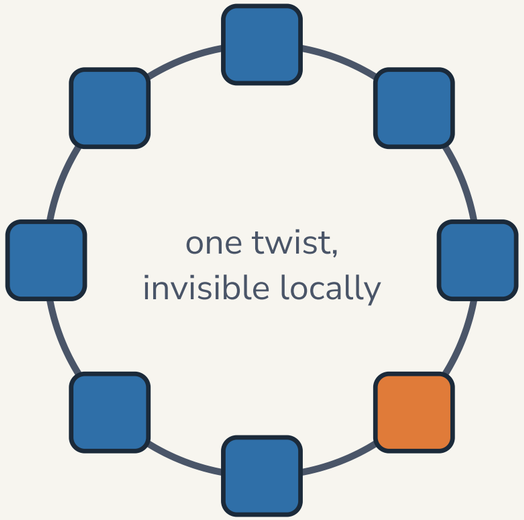
Individualise v: give v a colour of its own, then refine. A **base**: the sequence of vertices individualised. A **leaf**: every class is one vertex.

Branch over the vertices v could be: a **search tree**. Two leaves with the same colour pattern give a renaming, which is tested as an automorphism.

Known automorphisms prune branches (grey). This is how nauty, Traces, bliss and saucy work: extremely fast on most graphs.

COLOUR REFINEMENT

Graphs built to defeat the search



DEFINITIONS

A **CFI graph** (Cai, Fürer, Immerman 1992): replace each vertex of a base graph by a **gadget** and each edge by a pair of wires. A **twist** crosses one pair of wires.

Twisted and untwisted are not isomorphic, yet every gadget looks the same to refinement.

Multipedes (Neuen, Schweitzer 2018): rigid graphs on which every individualise-and-refine search is **exponential**.

In the benchmark: cfi-rigid (multipedes), sat_cfi_dim, chh, tnn.

PART 3 · THE FIRST PAPER

Refinement proposes, algebra decides

Refinement-derived algebraic computation of graph automorphism groups. Preprint: doi 10.5281/zenodo.23146211

REFINEMENT-DERIVED ALGEBRA · EXAMPLE

What is a candidate? Worked on the square

DEFINITION

Fix a class C and a base b in C. An image tuple s is **admissible** if, step by step, individualising s gives the same class sizes as individualising b. Its **candidate** maps each vertex of C to the vertex that ends with the same colour under s.

STEP 1 · class

All four vertices look alike: C = {a, b, c, d}.

STEP 2 · base (a, b)

Individualise a: {a}, {b, d}, {c}. Then b: all single. A leaf.

STEP 3 · images (x, y)

x: any of 4. y: must be a neighbour of x, 2 choices.

RESULT

4 × 2 = 8 candidates, here exactly the 8 symmetries.

One candidate in full: s = (c, d). Matching final colours: a → c, b → d, c → a, d → b. That is r^2, the half turn. ✓

REFINEMENT-DERIVED ALGEBRA

The candidate set T_C: never misses

DEFINITION

T_C = the set of all candidates of class C. The **restriction** of an automorphism to C is what it does on the vertices of C.

Always true

Every automorphism, restricted to C, is a candidate. T_C never misses a real symmetry.

Not always true

A candidate may be **fake**: fine on C, impossible on the whole graph. On CFI graphs $|T_C| = 2^k$ while $|Aut| = 1$.

Open question

Is T_C always a group? **Conjecture 11.1** of the first paper; proved in part 4 when admissibility is checked on all of V.

Precisely: $T_C = \{\tau_s : s \text{ admissible}\} \supseteq Aut(G)$ restricted to C, with equality when the leaves are discrete on all of V and admissibility is checked on V.

REFINEMENT-DERIVED ALGEBRA

Edges between classes become equations

DEFINITIONS

A group is **abelian** if order does not matter: gh = hg. **Mod 2** arithmetic: 1 + 1 = 0.

On a CFI graph each gadget has a bit x: 1 = flipped. The candidate groups are groups of bit flips: abelian.

The wires allow only some flip combinations. Those rules are **linear equations mod 2**. Gaussian elimination solves them and **counts** solutions without listing them.

TOY EXAMPLE

```
x1 + x2 = 0
x2 + x3 = 0
x1 + x3 = 0 (mod 2)
```

The third equation is the sum of the first two: rank 2. Three unknowns, so 2^{3-2} = **2 solutions**: flip none, or flip all three.

1,000 gadgets: one elimination, not 2^{1000} guesses.

Precisely: T_C is a set; $\langle T_C \rangle$ is the group it generates. The solver needs a group, so it checks at run time that $T_C = \langle T_C \rangle$ (it always was). Then a normal series of T_C with factors $(F_p)^k$ turns the edges between classes into affine systems over F_p, counted coset by coset: the classical tower method, without assuming bounded colour multiplicity.

REFINEMENT-DERIVED ALGEBRA

When T_C is too big: a stabiliser chain

DEFINITION

A **stabiliser chain** fixes roots r_1, r_2, … one at a time. At each level it finds the orbit of the next root among the symmetries fixing the earlier ones. Then |Aut(G)| = product of the orbit sizes.

Deciding an orbit

Is there a symmetry with r → x? Individualise r on one side and x on the other; the same algebra answers yes or no.

Choosing roots

Roots come from a new decomposition, **dialysis** (next slide), which cuts the graph into simpler parts.

Square

Roots a, b: orbit sizes 4, then 2, then 1. |Aut| = 4 × 2 × 1 = 8.

Precisely: an orbit query compares (G, r) with (G, x); the answer is an automorphism, verified edge by edge, or a proof that none exists.

REFINEMENT-DERIVED ALGEBRA · A REAL RUN

T_C and the chain on Johnson graphs

DEFINITION

The **Johnson graph** J(n, k): one vertex per k-element subset of {1, …, n}; two are adjacent when they share k − 1 elements. Refinement sees one class.

J(6, 3): 20 vertices

T_C of the one class: **1,440** candidates, a group, all real symmetries. Solved by the algebra alone in 0.28 s.

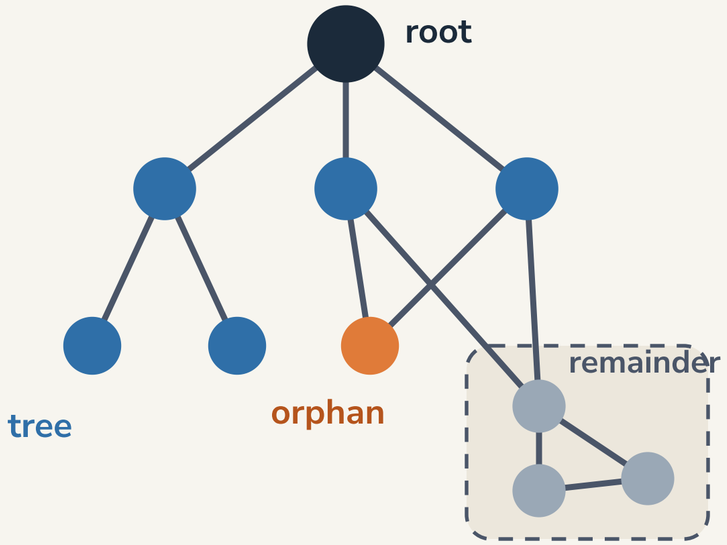
J(8, 3): 56 vertices

step	what happens	number
depth 0	T_C of the class: over 20,000 candidates	stop
fix a root r	4 classes: share 3, 2, 1, 0 elements with r	1, 15, 30, 10
depth 1	T_C groups + equations: stabiliser of r	720 = 3! × 5!
orbit of r	56 of 56 candidates, 1 query	56
|Aut|	56 × 720	40,320 = 8! ✓

Xeon, 1.2 s in total. Logs: results/xeon-2026-10/johnson (series.csv; chain/johnson-8-3.wl1.txt).

REFINEMENT-DERIVED ALGEBRA

Dialysis and the colouring Π



DEFINITION: DIALYSIS FROM A ROOT

Go out layer by layer. A vertex joins the **tree** if it has exactly one neighbour in the tree one layer up. An **orphan** has all its neighbours in the tree. What is left falls into **remainders**, each peeled again from its own root.

DEFINITION: THE COLOURING Π

Refine, then split classes by the shape of each vertex's dialysis tree and orphan count, then by its remainders' colourings. Invariant, at least as fine as 1-WL.

What the first paper achieves

12,001

instances of 36 benchmark families, all solved exactly

10,911

by the algebraic solver alone: no backtracking

0

disagreements with Traces wherever it answered

Proved polynomial

On **multipedes**, where every individualise-and-refine search is exponential. On all **Johnson** and **Hamming** graphs (they are Tinhofer graphs).

DEFINITION

A **Tinhofer graph** (Arvind, Köbler, Rattan, Verbitsky): after individualising any tuple of vertices, refinement's classes are exactly the orbits. Then the chain needs no search.

On projective planes that property fails; there the chain matches Miller's classical bound N^O(log log N).

PARTS 4 AND 5 · THE SECOND PAPER

When the hidden symmetry is not abelian

A practical graph isomorphism solver and an explicit quasi-polynomial bound. Preprint: doi 10.5281/zenodo.23458588

PRACTICAL SOLVER

Group-CFI: gadgets where order matters

DEFINITION

Group-CFI over Γ: like CFI, but wires carry elements of a group Γ instead of bits. A gadget allows (g_1, g_2, g_3) when $g_1 g_2 g_3 = 1$.

S_3 (6 symmetries of a triangle) and **A_5** (60 rotations of an icosahedron) are non-abelian: $gh \neq hg$. The rules are no longer linear equations.

S_3 is the smallest non-abelian group; A_5 is not even solvable: it cannot be broken into abelian layers.

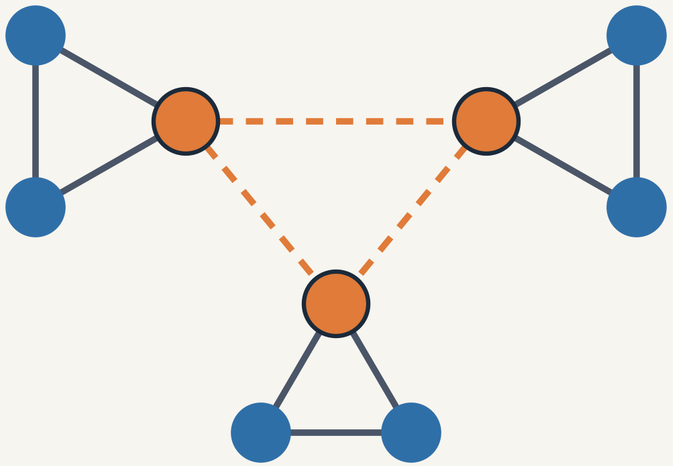
S_3 group-CFI on random cubic bases

base	vertices	Traces	ours
rc12	648	0.8 s, 2.8 s	2 s, 1 s
rc20	1,080	483 s, killed	1 s
rc24	1,296	killed	4 s
to rc64	3,456	not run	3 to 25 s

rcN = random cubic base with N vertices. Two times = untwisted, twisted. Killed without answer: rc20 twisted after 6,124 s, rc24 after 45 s.

PRACTICAL SOLVER

Cut along an invariant edge class



DEFINITIONS

An **edge class**: all edges whose ends have a given pair of colours. Every symmetry maps it to itself, so deleting it is an **invariant cut**. The parts left are **pieces**; isomorphic pieces share a **type**. The **quotient** has one node per piece.

Here refinement colours the apices (orange) apart from the rest (blue), so the wires (dashed) form one edge class. Cut it: three pieces, one type.

PRACTICAL SOLVER

The (D) route: pieces, then Luks

DEFINITION

P: all ways to apply a symmetry inside each piece and permute pieces of one type as the quotient allows. P contains Aut(G).

DEFINITION

Luks' method (1982): given a group P and a structure (here the cut edges), it finds exactly the members of P that preserve it, recursing over blocks. Fast while the pieces it splits into are small.

graph (Xeon run)	vertices	pieces	|Q|	|Aut(G)|	time	giant
S_3-CFI rc12	648	12	4	36	0.9 s	none
S_3-CFI rc12, twisted	648	12	4	12	0.8 s	none
A_5-CFI on K_4	15,120	4	24	57,600	77 s	degree 5

|Q| = order of the quotient's group. Pieces are solved by the same algorithm, recursively. "Giant" is defined in part 5.

PRACTICAL SOLVER · A REAL RUN

Luks on an S_3-CFI graph, step by step

gcfi-S3-rc12-1, 648 vertices. The cut removes **108 edges**: 12 pieces, all of one type, each with **1,296** symmetries. P acts on the 216 endpoints of the cut edges; Luks keeps the members that preserve those edges.

step	group left
start: P = all piece symmetries, $1{,}296^{12}$	2.2×10^{37}
refine 12 → 36 colours	1.0×10^{28}
13 orbit steps, each divides by 36	6.0×10^{7}
refine 42 → 44, four orbit steps	1,296
refine 62 → 64, two orbit steps	36

THEN THE QUOTIENT

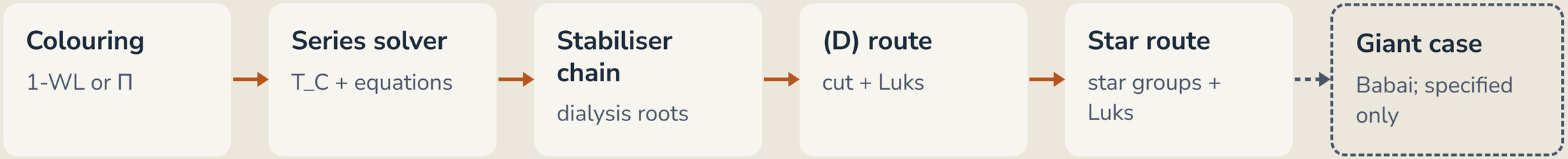
Q has order 4. Four isomorphism tests: no non-trivial element lifts. **|Aut(G)| = 36**, as Traces. Twisted graph: 12.

No giant. 0.89 s on the Xeon.

Log: results/xeon-2026-10-g2/luks_gcfi-S3-rc12-1.log. You run this graph yourself in the hands-on part.

PRACTICAL SOLVER

The whole solver: each stage is a fallback



DEFINITION

The **star group** S(C) of a class C: the automorphisms of C together with, for each other class, the sets of C-neighbours its vertices have. It contains every symmetry restricted to C, and replaces the cut when no edge class disconnects the graph.

A stage runs only when the previous one **stops**. Every stage is exact: an answer is never wrong, only sometimes missing. The dashed stage is specified and counted in the bound, but **not implemented**: no instance reached it.

When Luks runs: when the chain stops (time, work or candidate limit). In practice that means non-abelian local symmetry, where the edge rules are no longer linear.

PRACTICAL SOLVER · A VIEWER'S QUESTION

"Why not cut into pieces, run the pipeline on each, and skip Luks?"

We do run the pipeline on each piece. Luks does only what the pipeline cannot: the **gluing**.

Pieces: the pipeline

Each piece: series solver, then (D) route, then star route, recursively. How the pieces are joined: a graph too, solved for Q.

Not enough

S_3-CFI example: 12 pieces of 1,296 symmetries give $1{,}296^{12} \approx 2.2 \times 10^{37}$ combinations. Only **36** keep the cut edges.

Gluing: Luks

Which members of a **known group**, too big to list, preserve the cut edges? The pipeline answers questions about graphs, not this. Luks shrinks P: 10^{37} → 36 in 0.89 s.

Stars and balls: the same. The pipeline computes these ambient groups; cutting their product down to the true group is again Luks. Rerunning the pipeline on the whole graph is what had already stopped.

PRACTICAL SOLVER · A PROPOSITION

When the candidates form a group

DEFINITION

Admissibility on V: compare class sizes of **all** vertices, not only those of C, at every step of the candidate test.

DEFINITION

A **fractional automorphism**: a matrix $X \geq 0$, rows and columns summing to 1, with $AX = XA$ (A the adjacency matrix), only between equal colours.

PROPOSITION

With admissibility on V, the candidates of C are exactly the permutation blocks X_C of fractional automorphisms X. Products of such X are again such X, so **the candidates form a group**.

Why it matters: pruning with this group is now proved safe. Conditional answers became exact; S_3-CFI rc12 takes 7 s instead of a 1,800 s budget.

EXPLICIT BOUND

How fast can GI be solved in the worst case?

DEFINITION

Quasi-polynomial time: exp((log n)^c) for a constant c. With c = 1 it is polynomial; it grows far slower than any exp(n^ε).

1983

Babai, Luks: exp(O(√(n log n)))

2015

Babai: quasi-polynomial

2017

Helfgott: c = 3 suffices

2021

Dona: explicit constants

Babai works on **String Isomorphism**: given a group acting on positions and two strings, is there a group element turning one string into the other? GI reduces to it.

EXPLICIT BOUND

The one obstacle: giants

DEFINITIONS

Sym(m): all m! permutations of m points. **Alt(m)**: the even ones, half of them. A **giant**: a step of Luks' method where the group acts on m ≥ 5 blocks as at least Alt(m). The **degree** is m.

DEFINITIONS

A giant is **genuine** if the true symmetries of the string already act as at least Alt(m). Otherwise it is **loose**: symmetry the current group suggests but the graph lacks.

Why it is the obstacle

Luks' method branches over the blocks: with a giant of degree m it may face m! choices.

Genuine: example

Johnson graphs: their symmetry really is Sym(m) on the underlying points.

Babai's answer

Local certificates decide whether a giant is genuine, in quasi-polynomial time.

EXPLICIT BOUND

Counting the cost in bits

Bits = $\log_2$ of the number of branches the algorithm may explore. 10 bits = about a thousand branches, 20 bits = about a million. The second paper makes the leading constants explicit:

$11.4\,(\log_2 N)^3$

String Isomorphism: a factor of about 3.7 smaller than the published explicit analysis (Dona, 42.0, same units)

$43.4\,(\log_2 v)^3$

Graph Isomorphism on v vertices, with Babai's giant case as last fallback

$6.84\,(\log_2 v)^3$

if hypothesis (D_G) holds: no giant case needed

For v = 1,000 that is about 43,000 bits, or 6,800 under (D_G): a worst-case guarantee, not a running time you would ever see. The exponent 3 is Helfgott's; the polynomial work per branch is not made numeric.

EXPLICIT BOUND

Conjecture G2: the giant never really comes

Hypothesis (D_G), per branch

Either the chain **finishes cheaply** (as on projective planes), or every giant it meets is **genuine** or has degree at most m_0, small enough for brute force within the bound. $m_0 = \max(91, 10\lfloor \log_2 N \rfloor + 31)$, $N \le v^2$.

Conjecture G2

Every graph satisfies (D_G), using the colouring Π. Then Babai's giant case is never needed.

12,001

benchmark instances: giant case never reached

5 rounds

of graphs built to break G2: none did

26

largest giant met with Π, far below 91

EXPLICIT BOUND · A LOOSE GIANT

A loose giant is a question to the invariant first

Plain refinement

On a chh-type graph (1,408 vertices) it meets Sym(168): a giant that the graph does not really have.

The colouring Π

Its classes are exactly the orbits. The giant is gone, and the graph is solved in 13 s.

168 is below $m_0 = 231$ for this graph, so it is no counterexample, but it is six times the largest giant ever met with Π.

EXPLICIT BOUND

An open challenge: break G2

1	Find a graph on which, starting from Π and with recursion inside pieces, the chain does **not** finish cheaply,
2	and a **loose** giant of degree above m_0 appears for **every** allowed choice of ambient groups (the star or ball groups Luks starts from).
→	That refutes G2 as stated: this Babai-free argument is insufficient with Π. It does **not** show Babai's machinery is unavoidable: that needs a loose giant above m_0 surviving every polynomial-time invariant one can add.

TWO OPEN CONJECTURES

Perhaps the most valuable part

CONJECTURE 11.1 · FIRST PAPER

If the base discretises C, then **T_C is a group**, equal to the symmetry restricted to C of some graph G' that refinement cannot tell apart from G locally.

Known: no counterexample in any run; a theorem on multipedes; the group part proved for admissibility on V (second paper). **Open:** admissibility on C only, which can give a larger set, and round limits below stability.

Why it matters: refinement alone would produce groups, not just sets.

CONJECTURE G2 · SECOND PAPER

Every graph satisfies (D_G) with the colouring Π: **no branch ever needs Babai's giant case**.

Known: never needed on 12,001 instances or five rounds of attempts; largest giant met with Π has degree 26. **Open:** a proof, or a graph that violates (D_G).

Why it matters: bound 6.84 instead of 43.4 $(\log_2 v)^3$ bits, with no giant case.

EVIDENCE

Against Traces: honest in both directions

Traces is faster on 27 of the 36 benchmark families, often by a large factor.

On the families built against search it **stops answering** from some size on: 178 benchmark instances without an answer, all solved exactly by ours.

Same machine for both. Table: slowest answer of each solver below that size, on the same instances.

family	Traces slowest	ours slowest
cfi-rigid-t2	3,407 s	0.25 s
cfi-rigid-z3	1,739 s	2.9 s
chh	1,264 s	932 s
cfi-rigid-d3	530 s	3.9 s
sat_cfi_dim	344 s	1.6 s
cfi-rigid-s2	227 s	0.62 s
tnn	158 s	43 s

EVIDENCE

The hardest family: random class-2 groups

DEFINITIONS

The **Latin square graph** of a group G: one vertex per cell of its multiplication table ($|G|^2$ vertices), adjacent when two cells share a row, a column or an entry. **Class 2**: every commutator $ghg^{-1}h^{-1}$ commutes with everything. Random such groups have few automorphisms, hard to find.

group order	vertices	ours, look mode	Traces	|Aut| checked
128 (3 groups)	16,384	168 to 256 s	12 to 49 s	agrees with Traces
243	59,049	1,000 s	not run	independent check
256	65,536	6,468 s	not run	independent check

All on the Xeon. **Look mode**: stabiliser-orbit pruning and lookahead, both sound. Without it, the order-243 run was stopped by hand after 10,910 s. At 128 Traces is 4 to 14 times faster; at 243 and 256, $|\mathrm{Aut}| = |G|^2 \cdot 6 \cdot |\mathrm{Aut}\, G|$, with $|\mathrm{Aut}\, G|$ computed separately.

EVIDENCE · IN PRACTICE

Where the cost really comes from

never

was Babai’s giant case needed

26

largest giant with Π, against $m_0 \geq 91$

0.9 to 2.8

exponent b of the measured cost n^b, on the families measured

The quasi-polynomial terms of the bound stayed inactive; the cost came from refinement, the algebra and the chain’s refutation search.

WHAT IS PROVED ON THE HARD COMBINATORIAL FAMILIES

Quasi-polynomial bounds: n^O(log n) on Latin squares and Steiner triple systems (Miller’s bounds), and on Hadamard graphs; N^O(log log N) on projective planes. Measured: b ≈ 1.4 on Latin squares, 1.6 on Steiner systems; on planes the runs are budget-capped. Observations, not bounds.

HANDS-ON · STEP 0

Build the solver and make a graph

You need Linux, JDK 17 or newer, gcc and Python 3. The build wrapper caps itself at 4 GB; the examples need about 2 GB free.

```
git clone https://github.com/ckaratzas/dialysis-aut
cd dialysis-aut
scripts/run_gradle.sh paperJar

mkdir -p $HOME/aut-work
python3 scripts/paper4/gen_group_cfi.py $HOME/aut-work S3:rc12-1 S3:rc12-1:tw
```

Result: **gcfi-S3-rc12-1** and **gcfi-S3-rc12-1-tw**, S_3 group-CFI graphs on a random cubic graph with 12 vertices, untwisted and twisted, 648 vertices each. First line of each file: **p edge 648 1404**.

HANDS-ON · EXAMPLE 1

One graph through the whole flow

```
SERIES_TCPRUNE=1 SERIES_ADMIT_V=1 SERIES_TCGROUP=1 SERIES_AUTPRUNE=1 SERIES_GENAUT=1 \
SERIES_LUKS=1 SERIES_LUKS_PLAIN=1 LUKS_PIECE_S=600 LUKS_GLUE_MAX_TESTS=2000 \
ONE_PATH=$HOME/aut-work/gcfi-S3-rc12-1 ONE_DEADLINE_S=150 ONE_USE_PI=1 \
IR_FALLBACK=0 ONE_MAX_BASE=24 ONE_MAX_CAND=2000000 \
scripts/safe_java_test.sh ex1 1000 \
dialysis.experimental.DecompositionCanonicalFormsTest seriesOne $HOME/aut-work/ex1.log
```

END OF EX1.LOG (A LAPTOP RUN, 6.4 S)

```
TCGROUP |Aut| = 36
ONE .../gcfi-S3-rc12-1 original=true → EXACT |Aut⊨36.0000 ... (6.385s)
PEAK_ANON_MB 333
EXIT 0
```

HANDS-ON · EXAMPLE 2

An isomorphism test you can run

```
W=$HOME/aut-work; python3 scripts/paper4/gen_group_cfi.py $W S3:rc16-1 S3:rc16-1:tw
python3 scripts/paper4/disjoint_union.py $W/gcfi-S3-rc16-1 $W/gcfi-S3-rc16-1 $W/pair-iso 7
python3 scripts/paper4/disjoint_union.py $W/gcfi-S3-rc16-1 $W/gcfi-S3-rc16-1-tw $W/pair-noniso
ls $W/pair-* > $W/pairs.txt
INSTANCES_FILE=$W/pairs.txt SERIES_AUTPRUNE=1 SERIES_TCGROUP=1 SERIES_GENAUT=1 \
LUKS_PIECE_S=600 scripts/safe_java_test.sh ex4 1000 \
dialysis.experimental.DecompositionCanonicalFormsTest luksFullAut $W/ex4.log
```

pair-iso (G and a shuffled copy)

`|Aut(G)| = 3359232`

= 2 × 1,296², so **isomorphic**

pair-noniso (G and its twist)

`|Aut(G)| = 559872`

= 1,296 × 432, so **not isomorphic**

GLOSSARY

Every term, one line each

Automorphism: a renaming of the vertices mapping edges to edges.

Orbit / stabiliser: where a vertex can be sent / the symmetries that fix it.

Colour refinement (1-WL): recolour by neighbours' colours until stable.

Individualise: give one vertex its own colour, then refine.

Candidate: individualise a base b and admissible images s; map vertices of equal final colour.

T_C: all candidates of class C; contains every symmetry restricted to C.

Stabiliser chain: fix roots one by one; |Aut| = product of orbit sizes.

Series solver: T_C groups plus linear equations over F_p between classes.

Admissible on V: class sizes of all vertices match at every step; then candidates form a group.

Fractional automorphism: $X \geq 0$, rows and columns sum to 1, $AX = XA$.

Dialysis, Π: peel from a root into tree, orphans, remainders; Π is the colouring built from it.

Invariant cut: delete one edge class; what is left are pieces, joined by a quotient.

Luks' method: the members of a group that preserve a structure, found block by block.

Giant: a Luks step acting as at least Alt(m) on $m \geq 5$ blocks; genuine if real, else loose.

Bits: $\log_2$ of the number of branches an algorithm may explore.

(D_G), G2: each branch finishes cheaply or meets only genuine or small giants; G2 says every graph satisfies (D_G).

EVERYTHING IS PUBLIC

Read it, run it, try to break it

Refinement-derived algebra
doi.org/10.5281/zenodo.23146211

Practical solver and explicit bound
doi.org/10.5281/zenodo.23458588

Code, graphs, logs
github.com/ckaratzas/dialysis-aut

Christos I. Karatzas · ckaratza@gmail.com